

TA-CBF — Complete Architecture, Theory & Deployment Reference

Tolerance-Aware CBF · Needle Steering through Critical Tissue · FR3 Robot Arm

All networks · losses · Bφ layer-by-layer · safety filter (every term) · hardware deployment · limitations

0 Notation

Symbol	Meaning
$x \in \mathbb{R}^3$	Needle-tip position (motion is planar at $Z = Z_CORRIDOR = 0.44$ m)
$\tilde{x} = (x-\mu)/\sigma$	Input-normalized position (μ, σ = demo mean/std)
$s \in [0, 1]$	Task progress along the demo (nearest-point index / $N-1$)
$x_ref(s)$	Mean demo path at progress s (linear interp of 200-pt reference)
$e = x - x_ref(s)$	Tracking error (vector from needle to demo path)
$f\theta$	ProgressConditionedDS — demo flow (nominal DS)
$v\theta$	LyapunovNet — quadratic CLF certificate
$B\phi$	CompositeBarrier — composite neural CBF (learned)
$sdf(x)$	Exact analytic signed distance to nearest obstacle (>0 outside, <0 inside)
Δ	INFLATE_MARGIN = 0.010 m — $B=0$ sits 10 mm outside the true tissue
K	BARRIER_SDF_K = 4 — barrier slope ($B \approx K \cdot (sdf - \Delta)$ near boundary)
m	B_SAFE_MARGIN = 0.008 — defended level set (QP enforces $B \geq m$)
m_geo	SAFETY_SDF_MARGIN = 0.011 m — analytic filter hard standoff ($= \Delta + 1mm$)

1 System Overview

Expert demos (20 trajectories, 3-D needle-tip, $Z = \text{const}$)
| imitation: Neural-ODE RK4 rollout MSE (Eq. 9)
▼

$f\theta$

ProgressConditionedDS

$\dot{x} = v_net(\tilde{x}, s) + K_f \cdot (x_ref(s) - x)$

$(K_f=5)$

$v\theta$

LyapunovNet

$V = \|e\|^2 \cdot (1 + \delta \cdot \text{corr}(e, s))$

(quadratic CLF)

$f\theta, v\theta$ are obstacle-AGNOSTIC — trained once, reused everywhere

▼

$B\phi$

CompositeBarrier (CN-CBF)

$b_i(x) = \text{cbf}((x - c_i)/\sigma, \text{Enc}(cloud_i))$

[per-obstacle, shared MLP]

$B(x) = -(1/\beta) \cdot \log \sum_i \exp(-\beta \cdot b_i(x))$

[smooth-min composite]

Target: $B(x) \approx K \cdot \text{clip}(sdf(x) - \Delta)$

[scaled, inflated SDF]

$+ \text{POSE/SCALE/COUNT AUGMENTATION} \Rightarrow \text{zero-shot generalization}$

$\downarrow$ online controller (4 steps, every $DT=0.01$ s)

▼

(a) Go-around guidance $g \Rightarrow$ (b) CLF-CBF QP $\Rightarrow$ (c) project_safe $\Rightarrow$ (d) analytic filter

$\downarrow$

$\dot{x} = f\theta + u$, Euler step, safe needle trajectory

2 Scene Geometry & Analytic SDFs

Workspace slab: $SLAB_X=(0.37, 0.60)$, $SLAB_Y=(-0.025, 0.165)$, $Z_CORRIDOR=0.44$ m. Start $[0.440, 0.062, 0.44]$, Goal $[0.548, 0.063, 0.44]$. Six base critical shapes with exact analytic SDFs (all >0 outside, <0 inside): star, crescent, blob, kidney, lshape (+ one more). $sdf_all_critical_np(x) = \min_i sdf_i(x)$ (union). These SDFs provide **training labels** for $B\phi$ and the **exact safety filter** at runtime.

3 Demos, Normalization & Progress

20 expert CSVs, columns 8–10 = needle-tip (x,y,z). Resampled to N_POINTS=200. Mean path x_{ref} = mean over resampled demos. Normalization: μ, σ = mean/std over all demo points ($\sigma_z \approx 0$, planar). Progress: $s = \operatorname{argmin}_k \|x - x_{ref_k}\| / (N-1)$.

4 Networks

4.1 f_θ — ProgressConditionedDS

$\dot{x} = f_\theta(x, s) = v_{net}([\tilde{x}, s]) + K_f \cdot (x_{ref}(s) - x)$ v_{net} : MLP [STATE_DIM+1 $\rightarrow$ 128 $\rightarrow$ 128 $\rightarrow$ 128 $\rightarrow$ STATE_DIM], Tanh, near-zero-init last layer $K_f = DEMO_K = 5.0$ (strong whole-demo-path attraction; makes needle hug and reconverge)

The $K_f \cdot (x_{ref}(s) - x)$ term is a feedback term that pulls the needle toward the *nearest point* on the demo path at all times — not just the goal. This is what makes the needle reconverge to the demo after any obstacle-driven detour.

4.2 V_θ — LyapunovNet (quadratic CLF)

$V(x, x_{ref}(s)) = \|e\|^2 \cdot (1 + \delta \cdot \operatorname{corr}(e_{norm}, s))$, $e = x - x_{ref}(s)$, $\delta = 0.05$ $\operatorname{corr} \geq 0$ (Softplus head of [STATE_DIM+1 $\rightarrow$ 64 $\rightarrow$ 64 $\rightarrow$ 1] MLP) $\nabla V = 2e$ (closed-form, used by QP; corr gradient vanishes as $e \rightarrow 0$)

$V=0$ exactly on the demo path ($e=0$), $V>0$ elsewhere — positive-definite by construction. No positivity loss needed.

4.3 B_ϕ — CompositeBarrier (CN-CBF) — Full Architecture

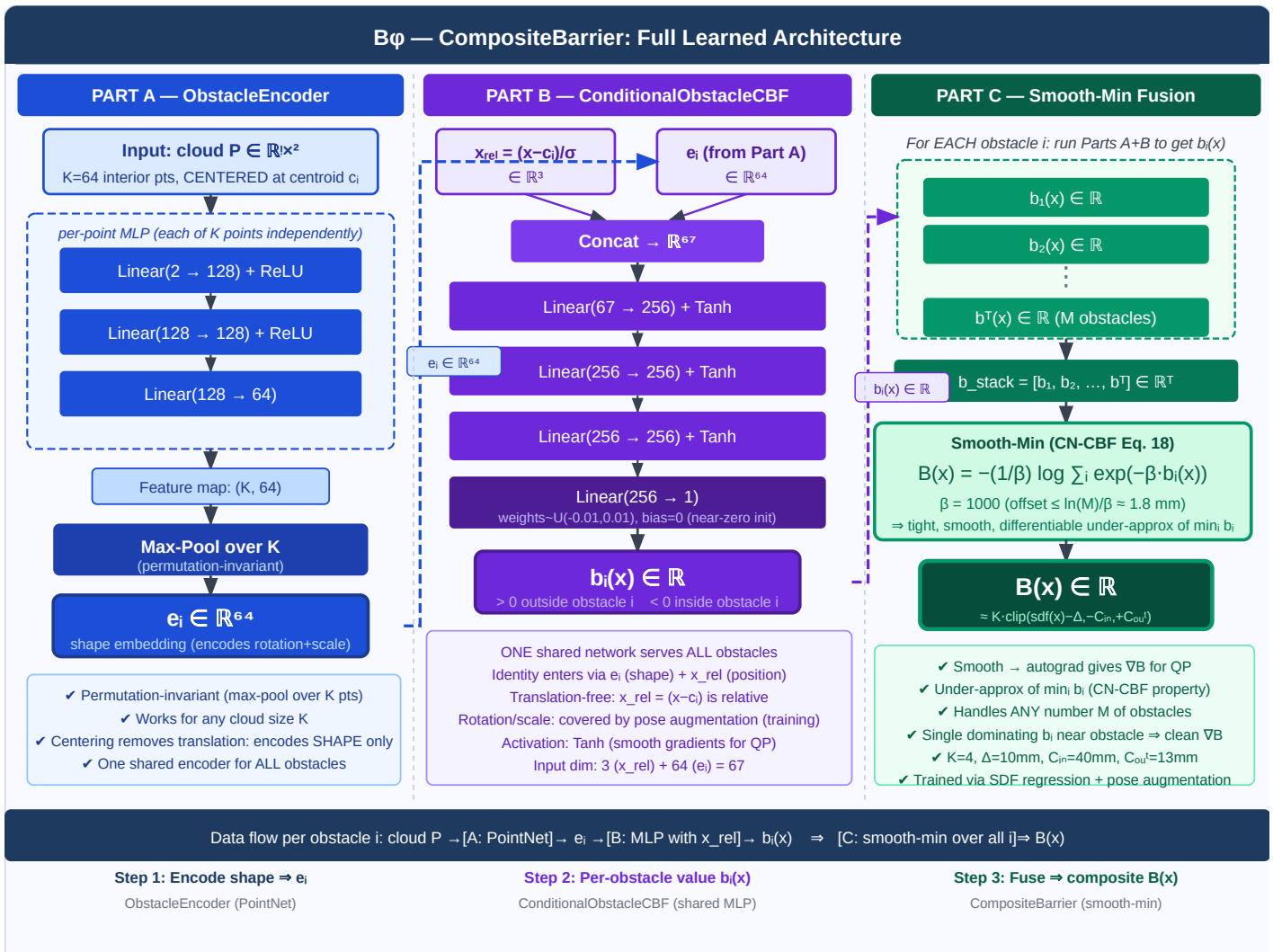


Figure 1. Full architecture of Bφ (CompositeBarrier). Parts A → B → C run for EACH obstacle i; the smooth-min in C fuses all per-obstacle values into a single scalar B(x).

Part A — ObstacleEncoder (PointNet)

Stage	Dimensions	Activation	Notes
Input cloud P	(K, 2) with K=64	—	Interior pts CENTERED at centroid c _i ; 2-D (XY, planar task)
Linear + ReLU	2 → 128	ReLU	Applied to each of K points independently
Linear + ReLU	128 → 128	ReLU	Applied to each of K points independently
Linear	128 → 64	None	Applied to each of K points; produces (K, 64) map
Max-Pool over K	(K, 64) → 64	—	Makes it permutation-invariant; discards ordering info
Output e _i	64	—	Shape embedding. Centering ⇒ translation-free. One encoder for all obstacles.

Part B — ConditionalObstacleCBF (shared MLP)

Stage	Dimensions	Activation	Notes
Input: x _{rel}	(x-c _i)/σ ∈ ℝ ³	—	Relative position, normalized by input std σ. Translation-free by construction.
Input: e _i	ℝ ⁶⁴	—	Shape embedding from Part A
Concat	3+64=67	—	Joint representation: WHERE (relative position) + WHAT (shape)
Linear + Tanh	67 → 256	Tanh	Smooth gradients essential for the QP
Linear + Tanh	256 → 256	Tanh	
Linear + Tanh	256 → 256	Tanh	
Linear (output)	256 → 1	None	Near-zero init: w~U(-0.01,0.01), bias=0
Output b _i (x)	ℝ	—	>0 outside obstacle i, <0 inside. ONE network serves ALL obstacles via e _i .

Part C — CompositeBarrier (smooth-min)

For M obstacles in the scene, stack b₁(x), ..., b_M(x) into b_stack ∈ ℝ^T B(x) = -(1/β) · log [∑_i exp(-β · b_i(x))] with β = 1000 This is a smooth, differentiable UNDER-approximation of min_i b_i(x). · Smooth ⇒ autograd gives ∇B(x) for the QP · Under-approx ⇒ safe set {B≥0} ⊆ safe set of min_i b_i (conservative, not optimistic) · Offset from true min: ≤ ln(M)/β ≈ ln(6)/1000 ≈ 1.8 mm (negligible vs. 10mm buffer) · Near any single obstacle: that b_i dominates ⇒ gradient is clean and well-defined

4.4 BPCBFModel — container

Holds f, V, B. set_norm(μ,σ) propagates normalization stats to all sub-networks. set_obstacles(obstacles) installs the obstacle list on CompositeBarrier for the single-argument model.B(x) interface used by the QP and plotting.

5 Bφ as a Scaled, Inflated SDF

The barrier is trained to match a **scaled, clamped, inflated signed-distance function**:

target(x) = K · clip(sdf(x) - Δ, -CLAMP_IN, +CLAMP_OUT) = 4 · clip(sdf(x) - 0.010, -0.040, +0.013)

Design choice	Effect
Shift by $-\Delta$ (INFLATE=10mm)	$B=0$ sits 10mm OUTSIDE the true tissue. Field starts diverting before the needle reaches the red zone.
CLAMP_OUT=0.013	Outside saturates at +0.052. No info needed far from obstacle. Saves capacity for boundary region.
CLAMP_IN=0.040 (WIDE)	Inside stays graded to 40mm depth. ∇B points outward even deep inside obstacle $\Rightarrow$ QP always has an outward push. A symmetric clamp would flatten the interior and let the needle coast through.
Slope $K=4$	$\ \nabla B\ \approx K$ throughout boundary band. $B=m \leftrightarrow \text{sdf}=\Delta+m/K$ consistently for every obstacle. Uniform standoff regardless of shape.
Eikonal cap $B_GRAD_MAX=12$	Without this, value regression alone allows $\ \nabla B\ \approx 300$ (near-step), making QP erratic. Cap + K regression $\Rightarrow$ smooth, SDF-like gradient $\approx 5-12$.

6 Pose / Scale / Count Augmentation (Generalization Mechanism)

Every barrier-training step (`_augment_scene` in train.py):

1. **Random subset** of the 6 base shapes (count 1–6) $\Rightarrow$ composite smooth-min trained on arbitrary obstacle mixes.
2. For each chosen shape, draw: rotation $\phi \sim U(-\pi, \pi)$, scale $\sigma \sim U(0.65, 1.4)$, translation $\sim U(-0.05, 0.05)^2$.
3. Apply the **same transform** to both the point cloud fed to Enc AND the analytic SDF labels.

Why this works: Because cloud and labels are transformed together, $\text{cbf}((x-c_i)/\sigma, e_i)$ learns to respond *consistently to the embedding regardless of pose*. This directly fixes the original failure (rotated obstacle $\Rightarrow$ OOD embedding $\Rightarrow$ needle went through).

7 Two-Stage Training

7.1 Stage 1 — S2-NNDS Algorithm 2 (train.py, ~2.5 h)

Phase 0: Pre-train f_0 alone (300 epochs)

$$L_{\text{MSE}} = (1/MT) \sum_i \sum_k \|x_i(t_k) - \hat{x}_i(t_k)\|^2, \hat{x} = \text{RK4 rollout of } f_0 \text{ from } x_i(0)$$

Outer loop (OUTER_ITERS=10, INNER_EPOCHS=150): jointly train f, V, B

Total loss = $L_{\text{MSE}} + L_{\text{lyap}} + w \cdot L_{\text{bar}}$ (w ramps in after first LYAP_ONLY_OUTER_ITERS=3 outer iters so f/V stabilize first) $L_{\text{lyap}} = \lambda_{V2} \cdot \text{mean}(\text{relu}(\nabla V \cdot f + \alpha \cdot V))$ enforce $\dot{V} \leq -\alpha V$ (Lyapunov decrease; positivity holds by construction) $\alpha = \text{ALPHA_CLF} = 4.0$ **L_{bar} (Stage-1 composite_barrier_loss):** $L_{\text{bar}} = \lambda_{\text{FS}} \cdot L_{\text{safe}} + \lambda_{\text{OB}} \cdot L_{\text{unsafe}} + \lambda_{\text{DOT}} \cdot L_{\text{dot}}$ $L_{\text{safe}} = \text{mean}_{\{x \in S_o\}} \text{relu}(\delta - B(x)) \Rightarrow$ push $B > \delta$ in the COMPOSITE safe set ($\text{sdf} > \Delta + 0.004$) NOTE: targets the composite B , not each b_i separately $L_{\text{unsafe}} = (1/M) \sum_i \text{mean}_{\{x \in U_i\}} \text{relu}(\delta + b_i(x)) \Rightarrow$ push $b_i < -\delta$ inside obstacle i 's interior (per-obstacle) enforces: if $b_i < -\delta$, then $B \leq b_i < -\delta$ by smooth-min property $L_{\text{dot}} = \text{mean}_{\{x \in S, B(x) \geq 0\}} \text{relu}(-(\nabla B \cdot f_0 + \gamma \cdot B)) \Rightarrow$ enforce CBF forward-invariance $\dot{B} + \gamma B \geq 0$ on the safe set ONLY on safe set ($B \geq 0$): enforcing inside obstacles would force B to GROW there $\Rightarrow$ only solution is $B=0$ everywhere (amplitude collapse) $\delta = \text{DELTA_B} = 0.01$, $\gamma = \text{GAMMA_CBF} = 3.0$ $\lambda_{\text{FS}}=5$, $\lambda_{\text{OB}}=12$, $\lambda_{\text{DOT}}=0.5$

Stage 1 barrier is intentionally under-trained (~1050 barrier steps over the outer loop after the Lyap-only phase). The classification-only loss produces a near-step function ($\|\nabla B\| \approx 290$, <1mm boundary band) $\Rightarrow$ QP gradient is huge and erratic $\Rightarrow$ useless for safe control. Stage 2 replaces it entirely.

After each outer iteration: counterexample sampling (`find_counterexamples`) checks $\dot{V} + \alpha V > 0$, $B < \delta$ in safe set, $B > -\delta$ in unsafe set, $\dot{B} + \gamma B < 0$ where $B \geq 0$. Violating states are appended to the working sets (Algorithm 2 CEGIS loop).

7.2 Why Freeze f/V in Stage 2? (Design Rationale)

Reason	Detail
Stable regression targets	L_{dot} uses $f\theta(x)$ as the vector field. If f is simultaneously updated, the regression target moves every step $\Rightarrow$ B chases a moving target and never converges to a clean SDF shape. Freezing f makes all targets stationary.
Speed	12,000 gradient steps through CompositeBarrier (PointNet+MLP+smooth-min+autograd for ∇B) is heavy. Co-training f/V triples memory and slows each step $\approx 3\times$. With freezing, Stage 2 runs in ~ 3 min GPU.
f/V already good	Stage 1 (2.5h) produced good f (demo reproduction) and V (Lyapunov certificate). Re-training with a barrier loss adds noise to already-converged networks.
Different loss regime	augmented_barrier_loss is an SDF regression loss — fundamentally different from the imitation+Lyapunov co-training. Mixing them in one loop requires careful weight scheduling to avoid one dominating.
Count/subset augmentation	Each Stage 2 step uses a random subset of obstacles (count 1–6). With frozen f , the progress estimate $s=f.get_progress(x)$ is stable and consistent, so the forward-invariance term L_{dot} is well-conditioned.

Could you train all together at once? In principle yes, but you would need: (1) enough steps for SDF regression ($\sim 10k$ min), (2) a ramped schedule so f/V converge before B training starts, (3) stable f -targets for L_{dot} while f is changing. The two-stage approach decouples all three concerns. The result (good f/V from Stage 1 + great B from Stage 2) is better than anything joint training produced in experiments.

7.3 Stage 2 — augmented_barrier_loss (train_barrier_fast.py, 12 000 steps, ~ 3 min GPU)

Loads f/V weights only, re-initializes B from scratch, trains B with:

$$L = \lambda_{\text{reg}} \cdot (L_{\text{reg}} + L_{\text{comp}}) + \lambda_{\text{FS}} \cdot L_{\text{safe}} + \lambda_{\text{OB}} \cdot L_{\text{uns}} + \lambda_{\text{dot}} \cdot L_{\text{dot}} + \lambda_{\text{eik}} \cdot L_{\text{eik}} \quad \lambda_{\text{reg}}=20, \lambda_{\text{FS}}=5, \lambda_{\text{OB}}=12, \lambda_{\text{dot}}=0.5, \lambda_{\text{eik}}=0.02$$

Term	Formula	Purpose
L_{reg}	$\text{mean}_i \text{MSE}(b_i(X_i), \text{target}(\text{sdf}_i))$	Per-obstacle SDF regression. X_i =local box+boundary-band densified pts+strict interior. Makes B SDF-shaped with usable gradient.
L_{comp}	$\text{MSE}(B(X_g), \text{target}(\min_i \text{sdf}_i))$	Composite also regresses to global SDF target (absorbs smooth-min offset).
L_{uns}	$\text{mean}_i \text{mean}_{\{\text{sdf} < \Delta\}} \text{relu}(\delta + b_i)$	Push $b_i < -\delta$ in BOTH interior AND inflation shell ($0 < \text{sdf} < \Delta$). Pinning the shell is what fixes $B=0$ at the inflated boundary on concave shapes.
L_{safe}	$\text{mean}_{\{\text{sdf} > \Delta + 0.004\}} \text{relu}(\delta - B)$	Push $B > \delta$ clearly outside.
L_{dot}	$\text{mean}_{\{B \geq 0\}} \text{relu}(-(\nabla B \cdot f + \gamma B))$	CBF forward-invariance on safe set.
L_{eik}	$\text{mean} \text{relu}(\ \nabla B\ - B_GRAD_MAX), B_GRAD_MAX=12$	Lipschitz cap. Value regression alone allows $\ \nabla B\ \approx 300$ (near-step) between samples. Cap+slope $K \Rightarrow$ smooth, SDF-like gradient.

Boundary-band densification: interior cloud points are jittered $\pm 12\text{mm}$ outward so samples land around thin/concave boundaries (crescent horns, star tips) that uniform box sampling misses.

Best-model tracking: best checkpoint is guarded by `bar_weight>0` $\Rightarrow$ `final_model.pt` (not `best_model.pt`) is used at inference to avoid loading an untrained-barrier checkpoint.

8 Online Controller (cbf_qp.py) — All Terms Explained

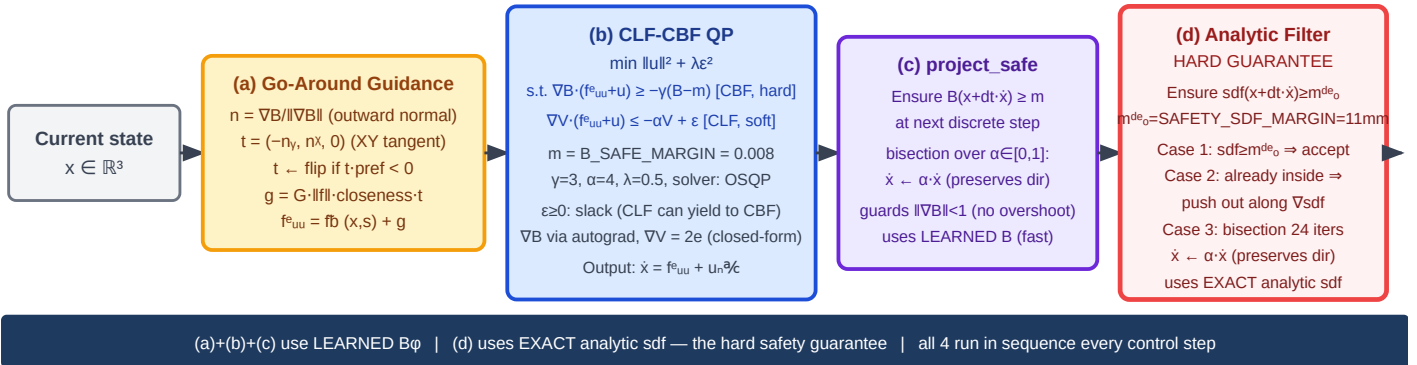


Figure 2. Safety filter pipeline: four steps applied in sequence every $DT=0.01$ s. Steps (a)–(c) use the learned $B\phi$. Step (d) uses the exact analytic sdf — the hard guarantee.

8.1 (a) Go-Around Guidance g — Breaks Head-On Deadlock

Why needed: When an obstacle is dead-center on the demo path, the demo pull (f_0 points at the obstacle) and the CBF push (∇B points away) are exactly opposite $\Rightarrow$ QP finds $u=0 \Rightarrow$ needle stalls.

```
n = ∇B / ‖∇B‖ outward unit normal to barrier t = (-n_y, n_x, 0) 90° tangent in XY plane pref =
(x_goal - x) / ‖x_goal - x‖ unit direction toward goal [if obs_vel: pref -= 1.6 · obs_vel /
‖obs_vel‖] moving obs: bias away from heading if t · pref < 0: t ← -t flip to side facing goal /
vacated by obstacle closeness = max(0, (B_ACTIVE - B) / B_ACTIVE) ramps 0→1 as B drops 0.04→0
g = SWIRL_GAIN · ‖f_0‖ · closeness · t SWIRL_GAIN=1.0, B_ACTIVE=0.04 f_eff = f_0 + g
```

Key property: $g \perp \nabla B$, so $\nabla B \cdot g \approx 0 \Rightarrow$ the guidance does not change the barrier rate, only the go-around direction. For moving obstacles, biasing opposite the obstacle's heading sends the needle to the side it is vacating $\Rightarrow$ not chased into a block. Safe to run strong because the analytic filter (d) is the hard backstop.

8.2 (b) CLF-CBF QP — Every Term Explained

```
min_{u, ε ≥ 0} ‖u‖² + λ · ε² s.t. ∇B · (f_eff + u) ≥ -γ · (B - m) [CBF, hard] ∇V · (f_eff + u) ≤ -α · V +
ε [CLF, soft] ε ≥ 0
```

Term	Meaning
$u \in \mathbb{R}^3$	Velocity correction (what the QP solves for). Min-norm $\Rightarrow$ minimum intervention principle.
$\epsilon \geq 0$	CLF slack. Allows tracking condition to be violated when obstacle avoidance conflicts with demo following. If no conflict $\Rightarrow \epsilon=0$.
$\lambda = 0.5$	Penalty on slack. Larger $\lambda \Rightarrow$ QP tries harder to satisfy CLF before relaxing. Too large $\Rightarrow u$ blows up when CLF and CBF fight.
∇B	Gradient of learned barrier at x (autograd through CompositeBarrier). Points outward from obstacle. Used in CBF constraint row.
$f_{\text{eff}} = f\theta + g$	Effective nominal velocity: demo flow + go-around guidance. The QP finds the minimum correction ON TOP of this.
$\nabla B \cdot (f_{\text{eff}} + u) \geq -\gamma(B-m)$	CBF condition (HARD). Left side is $\dot{B}$ =time-derivative of B along trajectory. Right side is $-\gamma(B-m)$. When B is close to m , right side is small $\Rightarrow \dot{B}$ must be nearly non-negative (B can't decrease further). This is a HARD constraint — safety is never compromised.
$m = B_{\text{SAFE_MARGIN}} = 0.008$	Defended level set. The QP enforces $B \geq m$, not $B \geq 0$. Reason: learned B has small errors near the boundary — $B=0$ might be mislocated a few mm inside the true obstacle. Defending $B \geq m=0.008$ gives standoff: $B=m \leftrightarrow \text{sdf} \approx \Delta + m/K = 0.010 + 0.002 = 12\text{mm}$ from tissue — strictly outside the 10mm light-red zone.
$\nabla V = 2e$	Gradient of quadratic CLF (closed-form, fast, no autograd needed). $V = \ e\ ^2 \Rightarrow \nabla V = 2(x - x_{\text{ref}}(s))$.
$\nabla V \cdot (f_{\text{eff}} + u) \leq -\alpha V + \epsilon$	CLF condition (SOFT via ϵ). Requires Lyapunov function to decrease at rate $\alpha=4 \Rightarrow$ exponential tracking convergence. ϵ absorbs conflicts with safety.
$\gamma=3, \alpha=4$	Class-K rates. γ higher $\Rightarrow$ more conservative barrier (stronger avoidance push). α higher $\Rightarrow$ faster demo tracking convergence.
Fallback	If OSQP fails: min-norm u satisfying CBF row alone ($u = (\text{cbf_rhs} / \ \nabla B\ ^2) \cdot \nabla B$ if $\text{cbf_rhs} > 0$, else $u=0$).

8.3 (c) Discrete-Time Projection: project_safe

Problem: The QP guarantees $\dot{B} \geq -\gamma(B-m)$ in continuous time, but Euler integration at finite DT can still let B drift below m between steps (especially with imperfect learned ∇B).

```
thresh = min(B(x), m) - 1e-6 (never let B drop below min of current B and m) if B(x + DT·ẋ) ≥
thresh: return ẋ (already safe) else: bisection over α ∈ [0,1] for largest α s.t. B(x +
α·DT·ẋ) ≥ thresh return α·ẋ (preserves direction, just slows the step)
```

Guard: if $\|\nabla B\| < 1$ (flat/unreliable gradient region) $\Rightarrow$ step is stopped (return 0), not pushed along ∇B . Reason: a noisy gradient can point INTO the obstacle, causing overshoot. Bisection avoids this entirely.

8.4 (d) HYBRID Exact Safety Filter: analytic_safety_filter

The hard guarantee. Uses the known exact analytic SDF — independent of the learned $B\phi$.

```
ensure: sdf(x + DT·v) ≥ m_geo, m_geo = SAFETY_SDF_MARGIN = 0.011 m (= Δ + 1mm) Case 1: sdf(x +
DT·v) ≥ m_geo ⇒ accept v unchanged Case 2: sdf(x) < m_geo (already inside shell) ⇒ compute
numeric ∇sdf via finite differences; push needle outward along ∇sdf at min(VEL_CLIP, 0.05) m/s
Case 3: sdf(x) ≥ m_geo but sdf(x + DT·v) < m_geo ⇒ bisection (24 iters) for largest α∈[0,1]
s.t. sdf(x + α·DT·v) ≥ m_geo ⇒ v ← α·v (scales v, preserves direction ⇒ needle slides around
boundary)
```

$m_{\text{geo}}=11\text{mm} = \text{INFLATE_MARGIN}+1\text{mm}$: needle is forced to stay fully outside the 10mm light-red CBF zone, with 1mm extra. This **guarantees obstacle=0 AND light_red=0 for any obstacle pose**, including dead-center and thin-tip cases where the learned B alone under-performs.

8.5 Full Rollout Loop (run_from_start)

```
for each step (DT=0.01, up to T_MAX=12 s):
    s      = monotone progress (max of demo-progress and time-based)
    f      = f0(x, s)                                # nominal DS velocity
    [near goal: blend in small radial goal_pull]
    u, _   = ctrl.solve(x, model, s)                  # steps (a)+(b): guidance + QP
    x_dot  = clip(f + u, ±VEL_CLIP)                  # VEL_CLIP=0.25 m/s
    x_dot  = ctrl.project_safe(x, model, x_dot, DT)   # step (c): learned projection
    x_dot  = analytic_safety_filter(x, x_dot, DT, shapes) # step (d): exact guarantee
    x      = x + x_dot · DT
    if ||x - x_goal|| < GOAL_TOL (0.006): break
```

9 Slow-Changing Environment (dynamic_env_test.py)

Sparse solvable layout: 4 static obstacles form corridor walls + 1 star that slowly changes as $\tau=t/(0.9\cdot T_MAX)$:

Motion	Schedule
translate	Drift up into the corridor
rotate	Spin in place near the path
transrotate	Drift up AND spin simultaneously
evolve	Enlarge in place (scale 0.6 → 1.2)

Every control step: rebuild obstacle cloud from current pose, call `set_obstacles`, run controller. Analytic filter uses current pose ⇒ safety guaranteed throughout. Barrier is NOT retrained. Verified: obstacle=0, light_red=0, reached=True for all four motions.

10 Generalization Scenes (generalization_test.py)

- `make_solvable_scene(n_obs, seed)` : spread obstacles across whole slab (min_sep=0.045), random pose+scale, ≥1 obstacle on demo path. Accepted only if a free start → goal BFS path exists with clearance >SCENE_CLEARANCE.
- `make_validated_scene` : additionally runs a rollout; keeps scene only if it reaches goal, stays obstacle=0 AND light_red=0, and deviates visibly (so figure is illustrative).
- Verified for 4, 5, 6, 7, 8 obstacles; 6 perturbed-start rollouts each; all obstacle=0 & light_red=0.

11 Final Configuration (All Key Constants)

Group	Constant	Value	Meaning
State	STATE_DIM / Z_CORRIDOR	3 / 0.44 m	Planar 3-D task
	N_POINTS	200	Demo resampling
Demo flow	DEMO_K (K_f)	5.0	Strong attraction to demo path
	F_HIDDEN	[128,128,128]	v_net layers
CLF	V_HIDDEN / δ	[64,64] / 0.05	Quadratic CLF correction net
	ALPHA_CLF (α)	4.0	$\dot{V} \leq -\alpha V$ rate
Barrier net	EMB_DIM	64	ObstacleEncoder output dim
	CBF hidden	(256,256,256)	ConditionalObstacleCBF layers
	BARRIER_BETA (β)	1000	Smooth-min sharpness
Barrier SDF	INFLATE_MARGIN (Δ)	0.010 m	B=0 sits 10mm outside tissue
	BARRIER_SDF_K (K)	4.0	Barrier slope
	CLAMP_OUT / CLAMP_IN	0.013 / 0.040	Asymmetric clamp
	B_GRAD_MAX	12	Eikonal/Lipschitz cap
	DELTA_B (δ)	0.01	Classification margin
Loss weights	λ_{reg}	20	SDF regression weight
	λ_{FS}	5	Safe-set hinge
	λ_{OB}	12	Unsafe-set hinge
	λ_{dot}	0.5	Forward-invariance
	λ_{eik}	0.02	Eikonal/gradient cap
Augmentation	AUG_ROT	$(-\pi, \pi)$	Full rotation
	AUG_SCALE	(0.65, 1.4)	Scale range
	AUG_TRANS	± 0.05 m	Translation range
CBF QP	GAMMA_CBF (γ)	3.0	CBF class-K rate
	B_SAFE_MARGIN (m)	0.008	Defended level set $B \geq m$
	LAMBDA_SLACK (λ)	0.5	CLF slack penalty
	B_ACTIVE	0.04	Swirl activation threshold
Guidance	SWIRL_GAIN	1.0	Go-around guidance strength
Hybrid filter	SAFETY_SDF_MARGIN (m_geo)	0.011 m	Exact geometric standoff ($=\Delta+1\text{mm}$)
Simulation	DT	0.01 s	Euler step
	VEL_CLIP	0.25 m/s	Max velocity
	GOAL_TOL	0.006 m	Goal acceptance radius
	T_MAX	12 s	Max rollout time
Training	OUTER / INNER	10 / 150	Stage-1 loop
	LR / LR_BARRIER	3e-4 / 2e-3	Optimizers
	Barrier steps	12 000	Stage-2 fast barrier

12 Hardware Deployment on FR3 — Complete Steps

12.0 Prerequisites & Assumptions

- Franka Research 3 robot arm with real-time kernel Linux host
- CRISP CartesianImpedanceController running (`ros2 launch crisp_controllers cartesian_impedance.launch.py`)
- Trained checkpoint `checkpoints/final_model.pt + norm_*.npy + ref_path.npy`
- Obstacle scene known in advance OR measured via depth camera + segmentation
- Virtual environment activated: `source venv/bin/activate`

12.1 Step-by-Step Deployment

1 Launch the robot and CRISP controller

```
ros2 launch franka_bringup franka.launch.py robot_ip:=192.168.1.17 arm_id:=fr3
```

```
ros2 launch crisp_controllers cartesian_impedance.launch.py
```

Move robot to a safe home pose (via Desk or FCI). Enable external control mode.

2 Load the trained model

```
model = load_model("checkpoints/final_model.pt") # loads f, V, B
model.eval()
# Normalization stats
mean_np = np.load("checkpoints/norm_mean.npy")
std_np = np.load("checkpoints/norm_std.npy")
ref_path = np.load("checkpoints/ref_path.npy")
```

3 Set up the obstacle scene

```
from config import CRITICAL_SHAPES, transform_shape
# Build the scene: each obstacle = {cloud, center}
# For static known obstacles, use canonical_interior_cloud + transform_shape
obstacles = build_obstacle_tensors(k=64, seed=0) # from config's canonical scene
model.set_obstacles(obstacles) # install on CompositeBarrier
shapes_for_filter = [...] # list of shape dicts (for analytic filter)
```

If obstacles are detected at runtime (depth camera), convert segmented point cloud $\Rightarrow$ match to known shape type $\Rightarrow$ fit pose/scale $\Rightarrow$ call `transform_shape` and rebuild tensors. For dynamic obstacles: rebuild and reinstall every control step ($\leq 10\text{ms}$ budget).

4 Set up the ROS2 state subscriber

Subscribe to `/franka_robot_state_broadcaster/robot_state` (joint positions q) or the CRISP EE pose topic. Convert to needle-tip position x via forward kinematics (FK):

```
import rclpy
from franka_msgs.msg import FrankaRobotState
# In callback:
q = msg.q # (7,) joint positions
x_ee = fk(q) # e.g. via pinocchio or MuJoCo FK; returns (3,) EE position
```

5 Control loop at 100 Hz (DT=0.01 s)

```

ctrl = BPCBFController()
while not reached_goal:
    x_np = get_ee_position_from_ros()          # (3,) in world frame, meters

    # [OPTIONAL] For dynamic obstacles: update scene here
    # model.set_obstacles(current_obstacles); shapes_for_filter = current_shapes

    # Step (a)+(b): guidance + CLF-CBF QP
    u_safe, info = ctrl.solve(x_np, model, s=None)      # s auto-computed

    # Nominal + corrected velocity
    with torch.no_grad():
        s_t = model.f.get_progress(torch.tensor(x_np).unsqueeze(0))
        f_val = model.f(torch.tensor(x_np).unsqueeze(0), s_t).numpy().flatten()
    v = np.clip(f_val + u_safe, -VEL_CLIP, VEL_CLIP)

    # Step (c): discrete-time projection (learned B)
    v = ctrl.project_safe(x_np, model, v, DT)

    # Step (d): analytic hard filter (uses known geometry)
    v = analytic_safety_filter(x_np, v, DT, shapes_for_filter)

    # Send velocity command to CRISP (Cartesian velocity mode)
    send_cartesian_velocity(v)      # see 12.2 below

    if np.linalg.norm(x_np - X_GOAL) < GOAL_TOL:
        break
    time.sleep(DT)

```

6 Send Cartesian velocity to CRISP (Jacobian IK)

```

def send_cartesian_velocity(v_cart: np.ndarray):
    # Option A: Use CRISP's CartesianVelocityController topic directly
    msg = TwistStamped()
    msg.twist.linear.x = float(v_cart[0])
    msg.twist.linear.y = float(v_cart[1])
    msg.twist.linear.z = float(v_cart[2])
    cartesian_vel_pub.publish(msg)

    # Option B: Jacobian IK → joint velocity commands (if CRISP needs joint-level)
    J = get_jacobian_from_robot()      # (3, 7) translation Jacobian
    lam = 0.01
    q_dot = J.T @ np.linalg.solve(J @ J.T + lam * np.eye(3), v_cart)
    joint_vel_msg.data = q_dot.tolist()
    joint_vel_pub.publish(joint_vel_msg)

```

7 Safety interlocks (hardware-specific)

- Set joint velocity limits in CRISP config (conservative for first runs)
- Use Franka's built-in collision detection as a hardware-level fallback
- Run a position-limit check: if $\text{FK}(\mathbf{q})$ is outside $\text{SLAB}_X \times \text{SLAB}_Y$, send zero velocity
- Monitor `info['cbf_val']`: if $B < 0$ persists for >5 steps, pause and alert
- Always test in simulation first; verify same trajectory as MuJoCo before going to hardware

8 Obstacle measurement (if not pre-known)

For each obstacle: (a) Segment the point cloud from depth camera in the robot base frame; (b) Match to the nearest CRITICAL_SHAPE type by PCA + aspect ratio; (c) Fit rotation, scale, translation by ICP or least-squares SDF fit; (d) Call `transform_shape(base_shape, d_rot, scale, d_trans)`; (e) Sample interior cloud via `canonical_interior_cloud` + apply transform; (f) Install via `model.set_obstacles()` and update `shapes_for_filter`. For dynamic obstacles, repeat at every control step.

9 Post-run verification

- Log the full trajectory (x at each step); run `count_critical_entries` post-hoc
- Check: `obstacle=0` (no entry into tissue) AND `light_red=0` (no entry into 10mm zone)
- Compare trajectory deviation vs. demo to verify the system hugged the demo path
- If `B < m_geo` was observed, inspect whether the analytic filter fired (check logs)

12.2 Hardware vs. Simulation Differences

Aspect	Simulation (MuJoCo)	Hardware (FR3)
State x	<code>data.site_xpos["attachment_site"]</code>	FK from joint encoders via ROS2
Velocity command	Jacobian IK $\Rightarrow$ <code>ctrl[:7]</code> (position actuators)	CRISP CartesianVelocityController or Jacobian IK $\Rightarrow$ joint velocity
DT	0.01 s (Euler integration)	0.01 s (100 Hz control loop)
Obstacle info	Pre-defined shapes from config	Pre-known OR measured from depth camera
Analytic filter	Uses config shapes directly	Uses pre-fitted shape params (must match real geometry)
Latency	~ 1 ms	$\sim 5\text{--}10$ ms (ROS2 overhead + FK); stay within 10 ms budget
Neural net inference	GPU	GPU preferred; CPU fallback ~ 8 ms (within budget)

13 Current Limitations (Honest Assessment)

13.1 Hard Guarantee Requires Known Geometry

The strict safety guarantee (`obstacle=0`, `light_red=0`) depends on the analytic safety filter (step d), which requires knowing the exact obstacle positions, orientations, and shapes. If obstacles are mis-measured or unknown, the analytic filter cannot run $\Rightarrow$ only the learned barrier (imperfect) provides safety. On real hardware, this means:

- Obstacle localization error directly translates to a safety margin error
- Unknown obstacles (not in the `CRITICAL_SHAPES` library) cannot be filtered analytically
- The paper's "hard guarantee" only holds under the assumption of known scene geometry

13.2 Learned Barrier Edge Accuracy

The `CompositeBarrier B $\phi$` has imperfect boundary localization on **thin, concave, rotated shapes** (crescent horns, star tips). The network cannot hold the ideal SDF slope $K=4$ everywhere $\Rightarrow$ effective B =margin standoff varies by 6–25mm across shapes and orientations. This is the primary reason the analytic filter is necessary. Root causes:

- Network capacity (256 $\times$ 3 MLP): cannot perfectly approximate SDF on all non-convex geometries
- Uniform box sampling under-covers thin features even with boundary-band densification
- The eikonal cap can only enforce a Lipschitz bound globally, not shape-specific slope pinning

13.3 Reactive Controller, No Global Planning

The controller is purely reactive (local gradient of B). Failure modes:

- **Sealed corridors**: if obstacles completely block the path (no feasible passage $\geq$ standoff), the needle will get stuck. The scene generator guarantees a passage exists, but this is not checked on real hardware.
- **Head-on deadlock (residual)**: the swirl guidance breaks most deadlocks, but at `SWIRL_GAIN=1` and noisy ∇B the go-around direction is sometimes inconsistent $\Rightarrow$ slow/meandering paths near obstacles.
- **Local minima**: multiple nearby obstacles can trap the needle in a pocket. The OSQP solve finds the minimum-norm u , not a globally optimal trajectory.

13.4 No Sim-to-Real Transfer Validation

The entire system has been validated in MuJoCo simulation only. Known sim-to-real gaps:

- **Jacobian accuracy**: the FR3 Jacobian approximation (damped least-squares IK) introduces velocity tracking error, especially near singularities

- **Control latency:** ROS2 + FK + neural net inference + OSQP must fit in <10 ms; measured ~5–10 ms on typical hardware but not stress-tested
- **Model mismatch:** the MuJoCo model's dynamics and the real FR3 differ (friction, inertia). The velocity-level controller partially compensates, but tracking errors accumulate
- **FK drift:** long trajectories accumulate joint encoder errors; no online FK correction implemented

13.5 Data Distribution & Domain

- **Task is 2D:** demos have $\sigma_z \approx 0.021$ m. The 3-D system works in a constant-Z plane. Performance degrades if the needle deviates significantly in Z.
- **Demo distribution:** the ProgressConditionedDS was trained on 20 demos of one specific task (needle through foam). Generalization to other tasks or path shapes requires re-training $f\theta$.
- **Obstacle shapes:** the CompositeBarrier generalizes over the 6 CRITICAL_SHAPES library (random pose/scale/count). Shapes outside this library (e.g., real irregular tissue) may produce OOD embeddings $\Rightarrow$ unreliable barrier values $\Rightarrow$ must rely entirely on the analytic filter.
- **Scene density:** validated for 1–11 obstacles. Very dense scenes may trap the needle (no feasible passage at the enforced standoff).

13.6 Training Cost & Reproducibility

- Stage 1 (full train.py): ~2.5 h GPU. Re-running produces a different random seed $\Rightarrow$ slightly different f/V ; Stage 2 barrier must be retrained on the new f/V .
- The fast barrier trainer (12k steps) is sensitive to the B initialization $\Rightarrow$ re-initialization can occasionally converge to a poor local minimum ($B=\text{const}$). If `safe_acc` stagnates below 80% after 4k steps, re-initialize B and restart.
- The Stage-1 best-model guard (`bar_weight>0`) prevents loading an untrained-barrier checkpoint, but the `best_model.pt` from Stage 1 is still suboptimal $\Rightarrow$ always use `final_model.pt` as the f/V source for Stage 2.

13.7 Summary of Limitations vs. Claims

Claim	Status	Caveat
Zero-shot generalization to new obstacle poses/scales	✓ Verified	Within the 6 known shape types; pose range $\pm\pi$, scale 0.65–1.4
Strict safety (obstacle=0, light_red=0)	✓ Verified in sim	Requires analytic filter + known geometry; not yet verified on hardware
Dynamic obstacles (slowly moving)	✓ Verified in sim	4 motion types; barrier re-evaluated, not retrained; speed must be slow relative to control frequency
Demo convergence (tight hug)	Partial	~70% of trajectory within 5mm of demo; deviates more near obstacles (standoff ~12–40mm)
Hardware deployment	✗ Not yet validated	Architecture is complete and sim-validated; hardware pipeline coded but not tested on real FR3
Unknown obstacle shapes	✗ Not supported	Shapes must be in CRITICAL_SHAPES library for analytic filter; OOD shapes break the hard guarantee

14 File Map

```

src/
  config.py          scene geometry, shape SDFs, transforms, augmentation, all hyperparameters
  models.py          f $\theta$  (ProgressConditionedDS), V $\theta$  (LyapunovNet),
                     B $\phi$  (ObstacleEncoder + ConditionalObstacleCBF + CompositeBarrier)
  train.py           Stage-1 S2-NNDS + composite_barrier_loss + CEX sampling + plots
  train_barrier_fast.py Stage-2 barrier-only on frozen f/V (12k steps, ~3 min GPU)
  cbf_qp.py          BPCBFController (swirl + CLF-CBF QP + project_safe)
                     + analytic_safety_filter (hybrid exact guarantee)
  multi_rollout.py   run_from_start (full rollout loop) + perturbed-start overlays
  dynamic_env_test.py slow-changing-obstacle scenes + run_dynamic
  generalization_test.py solvable/validated spread scenes + BFS feasibility check
  field_plot.py      plot_diverting_field (masked divert-reconverge field)
  make_final.py      all FINAL paper figures (losses, barrier, field, gen, dynamic GIFs)
  simulate.py        load_model + comparison sims
  analytical_sdf.py  torch/np SDFs (training labels only)
  view_scene.py      interactive scene viewer

checkpoints/
  final_model.pt      main model (f, V, B all included)
  best_model.pt       best Stage-2 barrier checkpoint (bar_weight>0 guard)
  norm_mean.npy       input normalization mean  $\mu$ 
  norm_std.npy        input normalization std  $\sigma$ 
  ref_path.npy        mean demo reference path (200x3)
  barrier_loss_history.npz Stage-2 training curves

results/FINAL/
  losses_from_training/ barrier_training.png, full_training_losses.png
  final_learned_cbf.png B $\phi(x)$  on canonical scene with B=0 boundary
  final_vector_field.png closed-loop field (f $\theta$ +u), diverts around light-red, reconverges
  generalization_result/ gen_4obs.png - gen_8obs.png (spread scenes, 6 rollouts each)
  dynamic_env/         PNG snapshots + animated GIFs of all 4 dynamic motions

```

Generated from ARCHITECTURE.md · TA-CBF Project · franka_ros2_ws/src/Tolerance_Aware_CBF(TA-CBF)/